Ingeniería y Calidad de Software

Resumen de conceptos para el 2.º parcial · Unidades 2, 3 y 4

Resumen de estudio propio, armado a partir de las slides de la cátedra y de resúmenes de estudiantes. Puede contener errores: contrastar con el material oficial. Quedan fuera Nexus y LeSS (confirmado por la cátedra).

1. Lean

- **Qué es:** filosofía nacida en Toyota (manufactura). Busca **eliminar todo lo que no agrega valor al cliente**. Es **menos prescriptiva que Ágil**: «tomá el proceso que ya tenés y mejoralo».
- **Desperdicio:** lo que no agrega valor al negocio; su tiempo debe tender a cero. **Tipo 1**: necesario pero sin valor directo → se **minimiza**. **Tipo 2**: puro desperdicio → se **evita**.

Los 8 desperdicios (7+1)

Manufactura	Software	Síntoma típico
Inventario	Trabajo parcialmente terminado	Muchas historias empezadas, ninguna terminada
Sobreproducción	Características extra	Funcionalidades que nadie pidió o usa
Proceso extra	Proceso extra	Burocracia, aprobaciones sin aporte
Transporte	Cambio de tareas / Transferencia de conocimiento	Saltar de tarea en tarea; pasar trabajo entre personas perdiendo información
Esperas	Esperas / Demoras	Esperar aprobaciones, entornos, respuestas
Movimiento	Cambio de contextos (seteo)	Tiempo de «volver a ponerse en tema»
Defectos	Defectos	Bugs que obligan a retrabajar
Talento no utilizado	Talento no utilizado	Gente con experiencia/ideas que no se aprovechan

Los 7 principios

Principio	Idea
Eliminar el desperdicio	No hacer lo que no agrega valor
Amplificar el aprendizaje	Aprender del trabajo y compartir el conocimiento en toda la organización
Embeber la integridad conceptual	Calidad desde el inicio; las partes encajan (revisiones entre pares)
Diferir compromisos	Decidir en el **último momento responsable**, con más información (ej.: no detallar el fondo del backlog)
Dar poder al equipo	Decide quien hace el trabajo (equipos auto-organizados)
Ver el todo	Visión del conjunto; evitar la optimización local de una parte
Entrega rápida	Ciclos cortos, incrementos pequeños de valor

2. Kanban

- **Qué es:** método para gestionar el **flujo de trabajo** y el cambio evolutivo **sobre un proceso que ya existe** («empezá por donde estás»). **No es** un ciclo de vida ni una metodología de gestión de proyectos. Viene del Just in Time de Toyota (*kan-ban* = tarjeta de señal).
- **Pull vs. push:** en pull se toma trabajo nuevo **solo cuando hay lugar**; en push te lo empujan aunque estés lleno. **WIP** = trabajo en curso. **Cuello de botella** = etapa donde se acumula el trabajo.

Las 6 prácticas

Práctica	Qué hace
Visualizar el flujo	Tablero visible para todos; columnas de producción y de acumulación («Hecho»)
Limitar el WIP	Máximo de ítems por columna; el trabajo nuevo entra cuando hay capacidad
Gestionar y medir el flujo	Tablero permanente; detectar cuellos de botella; Lead/Cycle Time
Políticas explícitas	Reglas visibles de cómo se mueve el trabajo (criterios de «terminado», WIP, etc.)
Ciclos de feedback	Revisiones periódicas a todos los niveles
Mejorar colaborativamente	Cambios graduales con experimentos: si no funciona, se vuelve atrás

Cómo se diseña un tablero

- **Columnas:** reflejan cómo trabaja hoy el equipo (izquierda → derecha); se pueden dividir en *En progreso / Hecho*.
- **Tipos de trabajo** (qué clase de ítem es): requerimientos, defectos, desarrollo/mantenimiento, solicitudes; con color y capacidad según la demanda. Justificar cada uno con el contexto.
- **WIP por columna:** ligado a la **cantidad de personas** de esa etapa. Justificarlo.
- **Clases de servicio** (cómo se trata el ítem; no es lo mismo que el tipo): **Expreso** (WIP = 1, **puede exceder** el límite, el resto espera) · **Fecha fija** (respeta el WIP; si se retrasa pasa a expreso) · **Estándar** (FIFO) · **Intangible** (sin urgencia).
- **Políticas:** reglas visibles, acordadas, pocas, sencillas y fáciles de modificar; **cada una con su propósito**. Ideas: criterios de «terminado» para pasar de columna, límites de WIP, reglas por clase de servicio, reposición de trabajo, cadencia de reuniones.

Lean ↔ Kanban (mapeo orientativo)

Principio Lean	Práctica de Kanban	Por qué
Eliminar desperdicio	Limitar WIP + visualizar	Menos trabajo a medias, esperas y cambio de tareas; se ve dónde se pierde tiempo
Amplificar aprendizaje	Feedback + mejora colaborativa	Se aprende y comparte mediante experimentos y revisiones
Integridad conceptual	Políticas explícitas	Calidad en cada paso (revisión de pares, Definition of Done)
Diferir compromisos	Pull	Se toma trabajo cuando hay capacidad, sin comprometerlo de antemano
Dar poder al equipo	Pull / autoasignación	Decide quien hace el trabajo
Ver el todo	Visualizar + gestionar el flujo	Flujo de punta a punta; evita la suboptimización local
Entrega rápida	Limitar WIP + medir Lead/Cycle Time	Ciclos cortos; clase Expreso para lo urgente

Scrum vs. Kanban

Aspecto	Scrum	Kanban
Tiempo	Sprints de tiempo fijo (1 a 4 semanas)	Flujo continuo; tiempo fijo opcional
Compromiso	Por sprint	Opcional
Roles	3 prescriptos (PO, SM, equipo)	No prescribe roles
Límite de WIP	Indirecto (lo marca el sprint)	**Directo, por etapa**
Trabajo nuevo	No durante el sprint (espera al siguiente)	Sí, cuando hay capacidad
Estimaciones	Obligatorias	Opcionales
Métrica por defecto	Velocity	Lead Time
Seguimiento	Burndown	No prescribe
Tablero	Se limpia en cada sprint	Persistente
Priorización	Backlog priorizado (PO, por valor)	Opcional

3. Métricas en los 3 enfoques

- Una métrica es un **número** que representa un atributo. **No se usan para evaluar personas**. Todas tienen un costo: se mide solo lo que sirve.

	Tradicional	Ágil	Lean (Kanban)
Qué mide	Proceso, proyecto y producto	El **producto**, por equipo	El **flujo** del proceso (sin proyecto)
Cuáles y quién	Muchas; se definen **al planificar** (líder de proyecto)	Pocas (3), definidas por el enfoque: «medir lo necesario y nada más»	4, definidas por el enfoque
Métricas	Tamaño, esfuerzo, tiempo, defectos (y costos)	Velocity, capacidad, RTF (+ burndown/burnup)	Lead, Cycle y Touch Time; eficiencia
Uso	Controlar desvíos contra el plan y estimar; las de proyecto se consolidan en métricas de proceso	Medir avance real y planificar el sprint; **no se comparan entre equipos**	Ver cuánto tarda en fluir el trabajo; detectar cuellos de botella y esperas

Definiciones y fórmulas

- **Dominios (tradicional): proceso** (toda la organización, públicas), **proyecto** (este proyecto, privadas, líder de proyecto), **producto** (lo construido). Las de proyecto se consolidan para armar las de proceso. Las 4 básicas: tamaño, esfuerzo (horas-persona), tiempo y defectos.
- **Velocity:** suma de puntos de historias **100% terminadas y aceptadas por el PO**; lo parcial o rechazado vale 0. Es propia de cada equipo (no extrapolable).
- **Capacidad:** *estimación* de lo que el equipo podrá completar el próximo sprint (horas ideales o puntos). **RTF** (Running Tested Features): funcionalidades implementadas y probadas, funcionando.
- **Lead Time:** desde que el cliente **pide** hasta que se **entrega** (incluye la espera). **Cycle Time:** desde que el equipo **empieza** a trabajarlo hasta que termina. **Touch Time:** tiempo realmente trabajado. Siempre Lead ≥ Cycle ≥ Touch.
- **Eficiencia = Touch Time / Lead Time × 100.**
- **Ejemplo:** pedido el día 0, el equipo empieza el día 3, entrega el día 8, 2 días efectivos → Lead = 8, Cycle = 5, Touch = 2, eficiencia = 2/8 = 25%.

4. Prácticas continuas

- Automatizar el camino del código hasta producción para detectar problemas temprano y desplegar con poco riesgo. Se encadenan: cada práctica requiere la anterior; **la diferencia es el nivel de automatización**.

Práctica	Qué es	¿Quién pasa a producción?
Integración continua (CI)	Integrar el código al repositorio compartido varias veces al día; cada integración dispara pruebas automáticas	No llega a producción
Entrega continua	CI + pruebas de aceptación automatizadas; el software está **siempre listo** para desplegar	**Una persona** (el PO) con un click
Despliegue continuo	Todo automático hasta producción mediante pipelines	**Nadie** (sin intervención humana)

- **Canary:** la versión nueva se lanza a un grupo chico de usuarios y se extiende si funciona. **Blue/Green:** dos entornos en producción; el tráfico pasa del viejo al nuevo y el viejo queda de respaldo. Ambas permiten actualizar sin cortes y revertir fácil.

5. Calidad: conceptos y aseguramiento

- **Calidad:** cumplir necesidades **manifiestas e implícitas**; es subjetiva. Aplica al producto, al proyecto y al equipo.
- **Síntomas:** atrasos y costos excedidos → calidad del **proyecto**; requerimientos poco claros o software que no hace lo que debe → calidad del **producto**.
- **QA (aseguramiento) es preventivo** (define procesos y estándares); **el testing es control** y llega tarde. **Más pruebas ≠ más calidad**; la calidad no se inyecta al final.
- El grupo de aseguramiento (GAC) debe ser **independiente** del equipo. Actividades de gestión de calidad: **aseguramiento** (estándares), **planificación** (plan de calidad), **control** (verificar cumplimiento).
- **Visiones de calidad:** usuario, producto, proceso, valor, trascendental. **Calidad programada** (planificada) / **necesaria** (mínimo para que sirva) / **lograda** (la obtenida): se busca que coincidan.
- **Proceso definido:** la calidad se asegura estandarizando el proceso. **Empírico:** depende de las personas del equipo. El proceso es el único factor controlable.

6. Modelos de calidad

- **Tres funciones: crear** el proceso (CMMI, ISO 9001) · **mejorarlo** (IDEAL, SPICE) · **evaluarlo** (SCAMPI, auditorías).
- **CMMI** (SEI): modelo **descriptivo** (dice *qué*, no *cómo*). **No se certifica: se evalúa (SCAMPI)**. Constelaciones DEV, SVC, ACQ. 22 áreas de proceso.

Nivel	Nombre	Característica
1	Inicial	Impredecible y reactivo («apagar incendios»); organización inmadura
2	Administrado	Controles básicos de gestión de proyectos; se recopilan métricas
3	Definido	Procesos estandarizados y documentados a nivel organización; proactivo
4	Cuantitativamente administrado	Procesos medidos y controlados con datos
5	Optimizado	Mejora continua

- **Por etapas:** mide **madurez** de la organización, 5 niveles, acumulativa, permite comparar. **Continua:** mide **capacidad** por área de proceso, 6 niveles (0 a 5), elegís qué mejorar.
- **Nivel 2 (foco de la cátedra):** REQM, PP, PMC, SAM, MA, **PPQA** (aseguramiento de calidad), **CM** (gestión de configuración).
- **IDEAL:** Initiating (sponsor) → Diagnosing (análisis de brecha) → Establishing (plan) → Acting (piloto) → Learning (extrapolar y aprender). **SPICE** (ISO 15504): mejora y evaluación, 6 niveles. **ISO 9001:** norma de calidad de proceso. **ISO 25010:** calidad de producto (teórica, no se certifica).
- **CMMI vs. Ágil:** CMMI mide y mejora el proceso (activos de proceso, mejora organizacional, predictibilidad, estabilidad); Ágil responde al cliente (conocimiento en las personas, mejora del proyecto, velocidad). Se pueden combinar.

7. Auditorías y revisiones técnicas

Auditorías

- Evaluación **independiente** (auditor externo) del cumplimiento de estándares y procedimientos con criterio objetivo. Se planifican: no son sorpresas.

Tipo	Valida que…
De proyecto	El proyecto **siguió el proceso** definido (según el PACS)
Config. funcional	El producto **cumple los requerimientos** (ERS, matriz de rastreabilidad)
Config. física	Lo construido **coincide con su documentación** (código vs. documentación)

- **Roles: auditado** (líder de proyecto; propone el plan de acción), **auditor** (externo; evidencia y reporte), **gerente de SQA** (plan de auditorías, costos, recursos, no conformidades).
- **Proceso (4 etapas):** preparación y planificación → ejecución (checklist, evidencia objetiva/subjetiva) → análisis y reporte → seguimiento (plan de acción y cierre).
- **Resultados: buenas prácticas** · **desviaciones** (requieren plan de acción) · **observaciones** (sin plan de acción). En ágil, la «auditoría» de proyecto la hace el propio equipo en la retrospectiva.

Revisiones técnicas

- Verificación y validación **estática** (no se ejecuta el programa) para detectar defectos temprano. Se revisa el **producto, no a la persona**.

	Walkthrough (recorrida)	Inspección
Formalidad	Informal	Formal, con checklists
Preparación	Poca o ninguna	Individual, previa
Roles	Un guía que recorre el producto	Autor, moderador, lector, anotador, inspector
Mediciones	No	Sí
Objetivo	Mínima sobrecarga, capacitar, retorno rápido	Detectar y remover todos los defectos
Extra	—	Reunión de máx. 2 hs; decisión: aceptar / rechazar / aceptar provisoriamente. No sirve para buscar soluciones, aprobar ni evaluar personas

	Revisión técnica	Auditoría
Quién	Pares técnicos	Externo e independiente
Qué evalúa	El producto de trabajo	El cumplimiento del proceso
Objetivo	Detectar defectos temprano	Verificar cumplimiento de estándares

8. Testing

- El testing es un proceso **destructivo**: busca defectos, no demostrar que funciona; un test es exitoso cuando **encuentra** defectos. El testing exhaustivo es imposible (se decide por riesgo y criterio de aceptación).
- **Error** (se halla en la misma etapa) vs. **defecto** (error no detectado que pasó a la etapa siguiente). **Severidad** (gravedad) vs. **prioridad** (urgencia): no siempre coinciden.
- **Paradoja del pesticida:** repetir los mismos casos deja de encontrar defectos nuevos. El tester no es enemigo del programador. Un programador no debe probar su propio código, salvo pruebas unitarias.

Nivel	Qué prueba	Quién	Ambiente
Unitario	Un componente aislado (fácil de automatizar)	Desarrollador	Desarrollo
Integración	Componentes ya probados, juntos y de forma incremental	Tester	Pruebas
Sistema	El sistema completo (funcional y no funcional)	Tester	Preproducción
Aceptación	Que el sistema sirve; da **confianza**, no busca defectos	Cliente / usuario	Preproducción o similar

- **Tipos:** funcionales y no funcionales (performance, carga, stress, usabilidad…). **Smoke** (lo básico funciona), **sanity** (tras correcciones), **regresión** (reverificar todo tras un cambio de versión).
- **Caja negra** (entradas y salidas): **partición de equivalencias**, **valores límite**; por experiencia: adivinanza de defectos, testing exploratorio. **Caja blanca** (estructura del código): cobertura de sentencias, decisión, condición, caminos básicos. Se **combinan**; el 100% de cobertura no garantiza ausencia de defectos.

	Testing tradicional	Testing ágil
Cuándo	Al terminar el desarrollo; es una fase	Continuo, en cada iteración; **no es una fase**
Quién	Tester con rol específico	**Todos** testean
Documentación	Plan de Prueba y casos detallados antes de ejecutar	Mínima; casos a medida que avanza
Automatización	Manual, automática o combinada	Unitarias, integración y regresión automatizadas; las **exploratorias no**
Enfoque	**Reactivo**: busca defectos	**Proactivo**: prevenir (TDD / ATDD); las pruebas son parte del **Done**

- **TDD:** la prueba se escribe **antes** que el código. Ciclo **Red** (test que falla) → **Green** (código mínimo para que pase) → **Refactor**. **ATDD:** igual, guiado por las pruebas de aceptación.
- **Cuadrantes del testing ágil:** 1 y 4 tecnológicos, 2 y 3 de negocio; 1 y 2 soportan al equipo, 3 y 4 critican el producto; el 4 son los no funcionales.

9. Práctico: partición de equivalencias y valores límite

- **Idea:** no se prueban todos los valores. Se agrupan los que el sistema trata igual (**clases de equivalencia**) y se prueba **uno por clase**. Con valores límite se eligen los **bordes**: el valor justo en el límite y los de al lado.
- **Condición externa ≠ clase:** «Cantidad» es la condición; «entero entre 1 y 6» es la clase.
- **Reglas:** rango → 1 válida + clases inválidas por debajo y por encima; dato obligatorio → «no ingresado» es **inválida**; dato opcional → «no ingresado» es **válida**; selección → una válida por opción + una inválida; existe / no existe; formato válido / inválido. Hacerlo para **entradas y salidas**, con **número único** por clase.
- **Casos de prueba:** uno **válido** agrupa muchas clases válidas; uno **inválido** lleva **un solo error**. Usar datos concretos y varios registros que pasen el filtro si se quiere verificar un orden.

Ejemplo de clases (cantidad de entradas, entero de 1 a 6)

Condición externa	Clases válidas	Clases inválidas
Cantidad	1. Entero entre 1 y 6	2. Menor que 1 · 3. Mayor que 6 · 4. No ingresada · 5. No entera
Tipo de entrada	6. General · 7. Jubilado	8. No seleccionado

Valores límite del ejemplo: **0, 1, 6 y 7** (o, con 6 valores: 0, 1, 2 y 5, 6, 7).

Plantilla de caso de prueba

ID	CE (clases cubiertas)	Nombre	Prioridad	Precondiciones	Pasos	Resultado esperado
Único	N.º de las clases (opcional)	Qué se prueba (éxito o error)	Alta / Media / Baja	Usuario, fecha, pantalla y **datos concretos** cargados	Acciones numeradas con **valores concretos**	Salida exacta con números y mensajes